abr. 15, 2026

Projetos Web API e Background Services

convidado Bruno Joaquim Cassia Dobler Fabio Diniz Francisco Schneider Gabriel Severo Gabriel kohlrausch Ruan Rodrigues aline.bianchini@t-systems.com ana.santos@t-systems.com analista.kuehn@outlook.com anderson.hilario@t-systems.com anderson.perim@t-systems.com andersongabriel_maria@hotmail.com anselmogabriel421@gmail.com bmf.pedro@gmail.com bruna.nery@t-systems.com bruno.destro@t-systems.com caique.rodrigues@t-systems.com d.silva@t-systems.com danilosilva2017@hotmail.com desenvolvedor@tidsoftware.com.br douglas.nishiyama@t-systems.com dpsnbsb79@gmail.com eduardo.kubota@t-systems.com eudy.amparo@t-systems.com fabio.marinelli@t-systems.com farley.t.i@hotmail.com felipeneves089@gmail.com gustavo.ferrazfontes@gmail.com hewerton.souza@t-systems.com i.parmigiani@t-systems.com igor.romulo.santos@gmail.com jeanderson.silva@t-systems.com jl.oliveir@gmail.com jonathan.jimenez@iatec.com karinetrevisann@gmail.com l.regioli@outlook.com leandro.piqueira@t-systems.com lucas.p.oliveira@outlook.pt lxsilvareis@gmail.com marcelohcordeiro@gmail.com marcio@almob.com.br marcos.ammon@gmail.com moraesguga@gmail.com n.segeti@t-systems.com nadson.dr11@gmail.com Paulo Scatena rafael.dias@t-systems.com regis.maioli@t-systems.com reinaldo.rodrigues@t-systems.com renan-silva@t-systems.com renan.baptista.cabral@gmail.com rodrigo_7sch@hotmail.com tatidornel@gmail.com thiago.fernandes@t-systems.com victor.prospero@t-systems.com victor.santos@t-systems.com victor.souza@t-systems.com vinicius.bonicio@t-systems.com webdeveloperrm@gmail.com wesley.boccaletto@t-systems.com wsilva@t-systems.com fabiano.felgas@gmail.com Thiago Borba

Anexos Projetos Web API e Background Services

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

Registros da reunião Transcrição Gravação

Resumo

Masterclass covered advanced ASP.NET architecture patterns with middleware, vertical slicing, and scalable multi-tenancy implementation strategies.

Arquitetura e Web API
ASP.NET Core utiliza middlewares e o servidor Kestrel com proxy reverso obrigatório. Minimal API oferece maior performance que Controller-Based para endpoints enxutos.

Serviços e Background Tasks
Injeção de dependência exige gerenciamento de escopo para evitar Captive Dependency. Background Service com Channel otimiza processamento assíncrono e resiliência.

Estratégias de Multitenancy
Segregação de dados utiliza middleware para identificação e contextos dinâmicos. Adoção de arquitetura de plugins e bibliotecas como LINQKit garante isolamento e escalabilidade.

Próximas etapas

- [ ] [Gabriel kohlrausch] Buscar Configuração: Procurar código exato de configuração para carregar controllers de assembly diferente.
- [ ] [Gabriel kohlrausch] Compartilhar Repositório: Compartilhar repositório contendo exemplos de código (modelos) da apresentação.
- [ ] [Thiago Borba] Criar Exemplo Channel: Desenvolver um exemplo de uso do Channel. Liberar código no GitHub para referência do grupo.
- [ ] [Gabriel kohlrausch, Thiago Borba] Conduzir Masterclass: Realizar a penúltima masterclass do treinamento sobre performance de aplicação e

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

recursos de concorrência. Incluir o desafio de serviço de envio massivo de emails.

- [ ] [Gabriel kohlrausch] Realizar Enquete: Fazer enquete para confirmar presença na reunião de feriado.

Detalhes

- **Início e Apresentações**: A reunião começou com saudações e uma breve conversa sobre a semana de trabalho dos participantes, incluindo Gabriel Canechia, que trabalha em uma software house de educação médica, a MED Group (00:00:00). Eles discutiram os desafios de reescrever um produto principal usando React Native e Next, mencionando a pressão e os prazos apertados, o que é comum na área (00:01:25).
- **Gerenciamento de Prazos e Dívidas Técnicas**: Gabriel Kohlrausch comentou que os prazos sempre serão apertados e que aceitar isso torna o trabalho mais fácil, apesar das dívidas técnicas que surgem no processo (00:01:25). Thiago Borba, mestre em IA na Eximia e especialista em .NET, juntou-se à discussão e foi apresentado (00:02:54).
- **Eventos Presenciais e Conexões Comunitárias**: Os participantes expressaram saudades da época de eventos presenciais do grupo .NET no Rio Grande do Sul. Gabriel Kohlrausch incentivou os participantes a convidarem a equipe da Eximia para participar de eventos em suas empresas, mostrando entusiasmo em trocar ideias e discutir assuntos (00:04:15).
- **Humor e Perspectivas de Fim de Semana**: A conversa incluiu um tom leve sobre o fim da semana de trabalho e a diferença de percepções de dias da semana, com um dos participantes sentindo que ainda era segunda-feira (00:05:00). Houve uma brincadeira sobre delegar tarefas para a IA, levantando a preocupação de que isso seria como delegar para um júnior sem revisão (00:05:52).
- **Adoção de IA e Vibe Coding**: Gabriel Kohlrausch expressou surpresa e desconcerto com Thiago Borba, que antes era cético em relação ao *vibe coding* com IA, e agora estava entusiasmado com um projeto *greenfield* (00:05:52). Thiago Borba confirmou que o *vibe coding* funcionou bem em projetos novos (*greenfield*), mas expressou ceticismo em relação à sua aplicação em projetos legados de dez anos (00:07:20).
- **Apresentação do Tópico da Aula**: Gabriel Kohlrausch deu as boas-vindas aos participantes para a última aula do primeiro módulo, que se concentraria em

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

ASP.NET Core Web API (00:08:18). Ele destacou que o desafio seria apresentar abordagens diferenciais e o que tem mudado o jogo em projetos dentro da Eximia, fugindo do básico sobre *controllers* e *actions* (00:09:23).

- **Importância da Participação e Escopo do Conteúdo**: Foi reforçado que a participação dos presentes é crucial para a troca de experiências e aprendizado, sendo quase obrigatória. O foco da aula seria Web API e *backend*, excluindo o Razor e *frontend* (00:10:40).
- **Funcionamento Base do ASP.NET Core (Kestrel e Middleware)**: O funcionamento do ASP.NET Core foi explicado, mencionando o uso do servidor HTTP Kestrel, que oferece alta performance (00:10:40). A requisição HTTP é encapsulada em um `HTTPContext` e transita por diversos *middlewares* em um padrão de *pipes and filters* (00:12:14).
- **Ordem de Execução do Middleware e Proxy Reverso**: A ordem de execução dos *middlewares* é 100% importante, sendo necessário ter atenção para evitar problemas, como a não efetividade de um *middleware* de tratamento de erros ou de log. É obrigatório não expor o Kestrel diretamente em ambientes de produção, exigindo a presença de um *proxy reverso* (como IIS ou Nginx) para lidar com recursos como *rate limit* e *offload* de SSL (00:13:55).
- **Cenários de Implementação Atuais**: Os participantes discutiram seus ambientes atuais de trabalho, que incluem uma mistura de aplicações legadas em full framework e aplicações mais novas em contêineres (00:17:54). Um participante mencionou o uso de .NET Core 6 e 9 no Azure, com a maioria das soluções sendo monolitos modulares em *App Service*, sem microsserviços (00:18:53).
- **Projetos Legados e WCF**: A discussão abordou a existência de aplicações legadas rodando em Windows Server antigo e usando WCF para comunicação entre sistemas como o BizTalk e o DB2 em *mainframe* (00:19:59). Essa menção gerou humor em relação aos desafios de memória e a necessidade de *IIS reset* (00:21:05).
- **Adoção de Minimal API**: A equipe questionou quem está usando Minimal API, e Igor Rômulo confirmou o uso misto em seus projetos (00:22:03). A única dificuldade encontrada por Igor Rômulo foi a integração manual com o DataDog, necessitando de um *middleware* customizado devido à forma como o Minimal API é estruturado (00:23:25).
- **Controller-Based vs. Minimal API (Opiniões e Uso)**: A visão da equipe é que *Controller-Based* continua sendo a melhor escolha para projetos grandes ou

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

monolitos complexos que exigem muitos recursos do ASP.NET, como filtros e versionamento complexo (00:24:41). Minimal API é mais adequado quando o design e a arquitetura estão muito bem estabelecidos, especialmente para *endpoints* simples e enxutos (00:28:49).

- **Simplificação Arquitetural (Vertical Slice e Custos)**: A conversa abordou a organização de projetos, questionando a necessidade de múltiplos *CS Projects* (Application, Domain, Infra) e introduzindo a ideia de Vertical Slice Architecture para diminuir o custo e o risco da mudança (00:26:07). Foi sugerido que é possível implementar Vertical Slicing com *Controller-Based* ao carregar *controllers* de outros *assemblies* (00:27:29).
- **Minimal API e Microsserviços**: Rafael Dias questionou se Minimal API faz mais sentido com microsserviços bem delimitados, ao que Gabriel Kohlrausch respondeu que, embora na teoria sim, na prática o ponto principal é garantir que o *endpoint* seja enxuto e não vaze responsabilidades, independentemente de ser um microsserviço (00:30:20). Igor Rômulo concordou, mencionando que a complexidade arquitetural é seu maior desafio, e que a arquitetura de *ports and adapters* pode ser uma solução (00:33:11).
- **Detalhes da Implementação (Controller-Based)**: Em aplicações *Controller-Based* padrão, a classe deve usar `[ApiController]` para facilitar o tratamento de resultados e o *binding* da requisição (00:35:25). A principal complexidade no versionamento não é o recurso do .NET em si, mas a governança e a política para manter as versões obsoletas funcionais. É recomendado trabalhar com *endpoints* assíncronos (`async/await`) para extrair o máximo de performance (00:36:49).
- **Detalhes da Implementação (Minimal API e Vertical Slice)**: A Minimal API pode ser organizada favorecendo uma estrutura de Vertical Slice, usando agregados em vez de camadas tradicionais (Domain/Application/Infra) (00:39:30). Um padrão de interface `IEndpoint` foi criado para facilitar a implementação do método `Map` e manter a perspectiva do *Minimum API* dentro de uma classe (00:40:50).
- **Vertical Slice Architecture e Atomicidade**: Marcelo Henrique buscou esclarecimento sobre o Vertical Slice, que sugere que cada *action* do *controller* se torne um *endpoint* (uma classe separada) para aumentar a atomicidade das mudanças (00:43:07). Essa abordagem consolida o contexto delimitado, movendo as classes de *endpoint* para perto do domínio (00:42:04).
- **Diferenciais de Performance do Minimal API**: Thiago Borba explicou que o Minimal API oferece vantagens de performance sobre o *Controller-Based*

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

devido à menor dependência de *reflection*. A Minimal API utiliza *delegate requests* compilados e tem menos serviços incluídos, o que permite a compatibilidade com o Native AoT e resulta em um ganho de performance (00:44:21).

- **Padrão Request Endpoint Response (RER)**: Foi apresentado o padrão RER (Request Endpoint Response) para trabalhar com Minimal API, focando na diminuição do acoplamento ao manter a request, o *endpoint* e o *response* juntos (00:46:37). Este padrão mitiga o risco de acoplamento, pois a explosão de alteração de um contrato está contida em um único arquivo (00:48:05).
- **Estruturas Baseadas em Volatilidade e Feature Folders**: Rafael Dias comparou o Vertical Slice com o conceito de *feature driven* ou *feature folders* do *frontend*, com o qual Gabriel Kohlrausch concordou ser similar (00:49:31). A discussão avançou para o conceito de arquitetura baseada em volatilidade, onde o design deve priorizar componentes com maior probabilidade de alteração (00:50:18).
- **Estrutura de Bootstrap e Logs**: O *bootstrap* de uma aplicação ASP.NET Core deve sempre incluir dois momentos distintos: configuração dos serviços (injeção de dependência) e configuração do *pipeline* de *middlewares* (00:53:29). Foi enfatizado o uso de `try/catch/finally` no `Program.cs` para garantir que exceções durante a inicialização sejam logadas e que o programa saia com `return 1` para facilitar a resiliência em ambientes orquestrados (00:55:41).
- **Uso e Flexibilidade do Serilog**: Gabriel Kohlrausch expressou sua preferência pelo Serilog para logs, destacando sua capacidade de carregar configurações de arquivos e enriquecer os logs com diversas informações. Além disso, eles apreciam a funcionalidade do Serilog que permite escrever logs para vários destinos diferentes (writes) (00:58:33).
- **Imutabilidade do Contêiner DI no .NET**: Thiago Borba mencionou que a partir do .NET 8, falhas silenciosas durante o startup da aplicação (no registro) foram corrigidas, resultando em "crash" da aplicação. Eles também destacaram que o método "Build" do contêiner de Injeção de Dependência (DI) é crucial, pois torna o contêiner imutável após sua geração, considerando qualquer manipulação posterior como uma "gambiarra" (00:58:33).
- **Práticas de Captura de Erros e Logs no .NET**: Gabriel Kohlrausch detalhou que para usar o Serilog, é necessário informar ao host para utilizá-lo como o motor de log principal, o que garante a integração com as interfaces de logger da Microsoft (00:59:46). Em resposta a uma pergunta sobre a necessidade de `try-catch` a partir do .NET 8, Thiago Borba recomendou manter o uso dessa estrutura para boas práticas e compatibilidade com

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

versões anteriores (.NET 6, 7), pois isso garante a captura de erros, o gerenciamento de logs e a garantia de `flush` (01:00:55).

- **Importância do Sinal de Retorno no Linux (Containers)**: Thiago Borba enfatizou que o `return 1` é importante para sinalizar um erro em ambientes conteinerizados, pois a ausência de um sinal de retorno fará com que o processo fique travado. Qualquer retorno acima de zero no Linux é um sinal de erro, e a ausência de um retorno adequado pode levar o orquestrador do contêiner (como o Kubernetes) a manter o pod ativo (01:01:59).
- **Impacto do Registro Singleton na Memória**: Lucas Pereira e Gabriel Kohlrausch definiram que um serviço registrado como Singleton resultará em uma única instância compartilhada por todas as requisições durante todo o ciclo de vida da aplicação. Gustavo Ferraz Fontes e Lucas Pereira notaram que, embora seja benéfico para objetos pesados que são custosos de gerar em cada requisição, um Singleton alocará e manterá o espaço de memória ocupado até o encerramento da aplicação, tornando-o potencialmente mais custoso (01:02:46).
- **Singleton e a Necessidade de Ser Stateless (Thread-Safe)**: Gabriel Kohlrausch considerou o registro de Singletons um ponto crítico, especialmente se a classe contiver grandes estruturas de dados, como listas ou arrays gigantes. Eles destacaram que se a instância é compartilhada por todas as requisições (e cada uma abre uma thread), o Singleton deve ser **stateless** (sem estado), pois o compartilhamento de estado pode levar a race conditions, exigindo locks e degradando a performance (01:05:12).
- **Casos de Uso Complexos para Singleton e Trade-offs**: Thiago Borba concordou que o Singleton deve ser muito bem planejado e justificou seu uso em casos complexos, como um pool de conexões com gerenciamento de memória e thread-safe, baseado em uma experiência anterior (01:06:49). Eles observaram que a complexidade da implementação deve ser muito bem justificada e que para estruturas de cache, a distribuição em balanceadores de carga já impõe o desafio da sincronização, exigindo casos de uso muito específicos (01:08:14).
- **Discussão sobre o Uso de Singleton como Cache em Desenvolvimento**: Eduardo Kubota compartilhou um cenário em que optou pelo uso de Singleton, implementando um `Concurrent Dictionary` para thread-safety e um controle de *lifetime* (expiração após duas horas) para dados com pouca mutabilidade (01:09:22). Eles adotaram essa abordagem como uma solução inicial, devido a limitações de infraestrutura (não usando Redis), com planos de migrar para soluções mais otimizadas, como Redis ou `IMemoryCache` futuramente (01:10:20).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Recomendação de Uso de IMemoryCache**: Gabriel Kohlrausch sugeriu a Eduardo Kubota o uso da interface nativa do .NET, `IMemoryCache`, que pode resolver o gerenciamento de *lifetime* e concorrência (01:11:41). Eles notaram que o `IMemoryCache` é nativo e seu uso facilitaria a evolução futura para um cache distribuído, como o Redis, que utiliza o `IDistributedCache`, exigindo poucas alterações na implementação (01:13:01).
- **Registro de Serviços Scoped e Transient**: Gabriel Kohlrausch mudou o foco para os níveis de escopo Scoped e Transient, e Gustavo Ferraz Fontes explicou que um serviço Scoped tem sua vida limitada à requisição, criando uma nova instância por requisição (01:15:07). Gabriel Kohlrausch complementou que o descarte (`dispose`) é automático pela requisição, citando o `DB Context` como um exemplo clássico de serviço Scoped (01:16:21).
- **Conceito e Perigo da Captive Dependency**: Gabriel Kohlrausch definiu um serviço Transient como aquele que gera uma nova instância a cada vez que é solicitado, mesmo dentro da mesma requisição (01:16:21). Eles alertaram sobre o problema da "Captive Dependency" (dependência cativa), que ocorre quando um serviço Singleton (com ciclo de vida longo) injeta uma dependência Scoped (com ciclo de vida curto), como um `ReportService` (Singleton) injetando um `DBContext` (Scoped), o que prende a instância Scoped no ciclo de vida do Singleton (01:17:56).
- **Validações de Injeção de Dependência no .NET**: Bruno Joaquim mencionou que a própria linguagem impede a injeção direta de um tipo Scoped no construtor de uma classe Singleton, prevenindo que a aplicação inicie (01:19:33). Thiago Borba esclareceu que essa validação não era padrão em versões antigas do .NET (o problema só aparecia em produção) e que essa validação da árvore de DI e dos ciclos de vida foi adicionada em versões mais recentes do .NET Core (01:20:58).
- **Boas Práticas para Retorno de Erros em APIs REST**: Gabriel Kohlrausch enfatizou que não se deve retornar o status HTTP 200 com uma mensagem de erro em APIs (01:23:08). A recomendação é usar os status de erro corretamente (ex: 400, 422 - Unprocessable Entity) e utilizar o padrão RFC para retorno de erro (`Return Problem`), que fornece um objeto padronizado com tipo, título, status code e a possibilidade de customização para detalhes do erro (01:24:23).
- **Segurança e Tratamento Global de Exceções**: Rafael Dias e Thiago Borba trouxeram a dica de segurança de ter cuidado com as informações retornadas no erro e no *stack trace* para evitar a exposição de dados

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

sensíveis ou informações internas da aplicação (01:25:30). Gabriel Kohlrausch recomendou o uso de um *handler* global de exceção (`UseExceptionHandler`) antes do `MapControllers`, pois isso substitui o uso desnecessário de `try-catch` em cada *controller*, garantindo um tratamento elegante de erros 500 (exceções) (01:26:36).

- **Uso e Vantagens do Background Service**: Gabriel Kohlrausch iniciou o tópico de *Background Service*, descrevendo-o como um recurso subutilizado que permite executar tarefas e processos em *background* dentro do processo da API, como *pullings* de banco, envio de e-mail ou consumo de filas (01:29:07). Lucas Pereira e Farley Souza confirmaram o uso de *Background Service* para consumo de filas e para verificar e atualizar status de integrações periodicamente (01:39:58).
- **Desafios de Escalabilidade e Concorrência (Leader Election)**: Rafael Dias levantou a questão crítica de que escalar uma aplicação que usa *Background Service* horizontalmente (múltiplas instâncias) pode levar à execução duplicada de atividades e problemas de concorrência (ex: gravar o mesmo arquivo duas vezes) (01:42:47). Gabriel Kohlrausch e Rafael Dias concordaram que, nesses casos, é necessário implementar estratégias de *leader election* ou *lock* para controlar a *race condition* (01:44:00).
- **Hosted Service vs. Background Service e Captive Dependency no Background**: Gabriel Kohlrausch explicou que a forma mais simples de implementar é herdando de `BackgroundService`, que gerencia o ciclo de vida (start/stop), diferente de `IHostedService`, que exige controle manual de threads e `dispose` (01:45:15). Para resolver o problema de **Captive Dependency** (injetar um Scoped em um Singleton, que é o caso do *Background Service*), a boa prática é injetar o `IServiceScopeFactory` e criar um escopo explicitamente antes de usar serviços Scoped, simulando o escopo de uma requisição (01:46:59).
- **Estratégias de Paralelismo e Uso de Channel**: Thiago Borba ressaltou a importância de controlar o escopo no *Background Service* e sugeriu a paralelização de chamadas de IO (por exemplo, ao processar um grande número de itens em *batch*) para otimizar o desempenho (01:51:28). Eles mencionaram o uso de técnicas como *producer/consumer* usando o objeto nativo `Channel` para controlar o fluxo de informação e *backpressure* no paralelismo (01:52:37).
- **Pedido de Exemplo de Channel e Limitação de Requisições**: Gabriel Kohlrausch desafiou Thiago Borba a criar um exemplo de uso de *Channel* para o GitHub, destacando que essa estratégia pode ser usada para comunicação entre processos, permitindo que uma API receba uma

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

requisição, use o *Channel* para iniciar um processamento demorado em *background* e libere a requisição rapidamente (01:53:39). Eduardo Kubota perguntou se o *Channel* poderia ser usado para limitar o número de requisições que entram em um *endpoint* (dosagem de entrada), mas Thiago Borba esclareceu que o *Channel* não se encaixaria nesse problema, que parece ser mais relacionado ao design da aplicação ou a problemas de *lock*, pois o ASP.NET já possui uma fila interna para requisições (01:54:55).

- **Discussão sobre Estratégias de Processamento Assíncrono e Throttling**: Eduardo Kubota levantou preocupações sobre o processamento custoso, especificamente devido à quantidade de chamadas ao banco de dados e ao processamento de informações que requerem consultas a APIs externas antes da persistência (01:57:27). Thiago Borba argumentou que, mesmo com chamadas custosas, se a aplicação utilizar estratégias assíncronas do começo ao fim (como a estratégia AD mencionada), eles não deveriam enfrentar problemas de *throttling* de requisição ou bloqueio de CPU/thread, a menos que haja problemas no desenvolvimento da aplicação (01:58:19). Gabriel kohlrausch sugeriu que a requisição não precisa ser síncrona, e pode-se devolver um `202 Aceito` e enfileirar o processamento, utilizando uma estratégia de *back pressure* (01:59:27).
- **Masterclass Futura sobre Performance em Aplicações**: Gabriel kohlrausch propôs a Thiago Borba realizarem uma masterclass juntos, focada em *channels*, recursos de performance em aplicação, e tópicos de ponta em desenvolvimento, como uma das últimas sessões do treinamento (01:59:27). Thiago Borba aceitou o desafio e sugeriu o tema de um serviço de envio massivo de e-mails, com restrições de API para simular um alto volume de requisições. O desafio para a masterclass foi aceito e marcado, o que envolve criar um serviço de envio massivo de e-mails com restrições na API (02:00:40) (02:46:54).
- **Apresentação de Ferramenta para Workflow Assíncrono**: Gabriel kohlrausch compartilhou o uso de uma ferramenta que permite a criação de *workflows* e o controle de estado, facilitando a resiliência ao retomar o processamento de onde parou em caso de falha (02:01:33). Eles mencionaram que a ferramenta é uma alternativa mais leve e fácil de usar do que o Hangfire, sendo utilizada para iniciar um *workflow* assíncrono após a API receber uma requisição e devolver a resposta imediatamente (02:03:02). Farley Souza comparou a ferramenta com o Hangfire, e o palestrante confirmou que a ferramenta é mais leve, sendo útil em cenários de orquestração e *saga* (02:01:33).
- **Controle de Processamento com APIs Externas**: Rafael Mendonça perguntou sobre a possibilidade de usar a ferramenta para controlar o processamento

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

que depende do resultado de APIs externas, o que foi confirmado por Gabriel kohlrausch. Eles explicaram que cada etapa do *workflow* é uma classe, e o processamento pode ficar preso em uma etapa até que o serviço externo devolva a resposta. O recurso é flexível, permitindo processamento paralelo, laços, condicionais (if/branches), agendamento em *background* e espera por eventos externos (02:04:25).

- **Conceito e Implementação de Multitenancy**: Gabriel kohlrausch introduziu o conceito de multitenancy, onde uma única aplicação atende a clientes diferentes, o que exige lidar com a segregação de dados (02:05:33). Existem duas estratégias principais: banco de dados único com uma coluna obrigatória para o cliente, ou segregação de dados com um banco de dados (database) exclusivo para cada cliente (02:06:55). A complexidade reside em controlar a segregação de dados e, em casos mais agressivos, criar um DB Context em tempo de execução de forma dinâmica (02:07:59).
- **Uso de Middleware para Gestão de Tenant**: O problema de duplicar a lógica de identificação do *tenant* em todo *controller* pode ser resolvido com a utilização de *middlewares* no *pipeline* da aplicação. O *middleware* deve extrair o *tenant* do contexto HTTP (por exemplo, do *header*, *subdomínio* ou *path*) e criar um objeto padrão para que toda requisição tenha essa informação (02:07:59). Para acessar as informações do *tenant*, é utilizada uma classe *accessor* que fornece acesso seguro ao objeto do *tenant* registrado no *middleware*, mantendo o registro de injeção de dependência imutável (02:09:20).
- **Segurança e Validação do Tenant**: Thiago Borba enfatizou a importância da segurança, pois o dado do *tenant* vem do *front-end* e precisa ser validado para garantir que o usuário autenticado pode acessar aquele *tenant*. Essa validação é crucial, pois, se o dado for apenas um `where` em uma *query*, um atacante poderia mudar o *tenant* no *header* e acessar dados de outros usuários (02:14:05). Anderson Perim complementou, dizendo que o *tenant* deve ser identificado por um sistema interno (como um *identity manager*) e não apenas pela requisição externa, para garantir que o usuário tenha permissão (02:15:20).
- **Estratégias de Segregação de Dados e Filtros de Query**: Gustavo Ferraz Fontes e Gabriel kohlrausch discutiram a injeção do *tenant* em uma *query* global para garantir a segregação de dados em um *single database*, o que é um recurso suportado pelo Entity Framework, embora exija cuidado ao usar consultas manuais (como com Dapper) (02:10:38). Anderson Perim recomendou o uso da biblioteca LINQKit para montar *predicates* e injetá-los no filtro de *query* de forma genérica, o que facilita o isolamento de dados e o cumprimento de *Row-Level Security* (RLS) (02:16:16) (02:18:00). Gabriel

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

kohlrausch concordou com a sugestão e enfatizou que o uso do LINQKit permite construir *queries* complexas de forma rastreável e performática (02:20:46).

- **Implementação de Multitenancy com Service Locator e DB Context Dinâmico**: Gabriel kohlrausch detalhou a implementação do *middleware* para multitenancy, que primeiro extrai a chave do *tenant* e depois usa um serviço *locator* para validar se o cliente é válido e possui licença (02:24:13). Para clientes com *databases* separadas, é implementado um *middleware* que cria um DB Context dinâmico em tempo de execução, usando uma *factory* de conexão que busca a *string* de conexão específica para aquele *tenant* (por exemplo, em um *Vault* ou serviço HTTP). Todos os componentes da aplicação podem então injetar um *accessor* que lhes fornece o DB Context correto para aquela requisição (02:26:50).
- **Desafios de Manutenção de Migrations em Múltiplas Bases de Dados**: Farley Souza questionou a dificuldade de manter as *migrations* em um cenário com múltiplas bases de dados por *tenant*, e Gabriel kohlrausch confirmou que é um *trade-off* que exige uma estratégia de sincronização de *updates* de *databases* ao liberar uma nova versão (02:29:44). A segregação de bases é necessária em cenários com grandes volumes de dados ou para requisitos de *compliance* que impedem o compartilhamento de dados entre clientes (02:30:59).
- **Estratégia de Extensibilidade e Plugins para Customizações**: Em vez de customizar a base da aplicação para cada cliente, a estratégia adotada foi a de extensibilidade, criando uma arquitetura de *plugins* que são *self-containing*. Essa abordagem permite que *plugins* específicos lidem com as particularidades de cada cliente sem alterar a estrutura base, mantendo-a única e reduzindo a complexidade (02:32:06).
- **Impacto na Performance e Estratégias de Escalonamento em Multitenancy**: Bruno Destro levantou a preocupação de que em um cenário de multitenancy, a alta concorrência de um único cliente (como um grande upload) possa fazer com que outros clientes paguem o preço da lentidão da API, mesmo que tenham bases de dados separadas (02:37:10). Gabriel kohlrausch confirmou que o recurso da API é compartilhado, e a solução é o escalonamento, que deve ser gerenciado com estratégias de DevOps e infraestrutura, como o uso de um *API Gateway* para rotear requisições de clientes específicos para serviços exclusivos (02:38:28).
- **Segurança e Autenticação com Múltiplos Esquemas**: A aplicação pode suportar múltiplos esquemas de autenticação, como JWT e API Key, implementando *authentication handlers* e definindo políticas dinâmicas

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

com base no *request* (02:34:33). Isso permite que o *middleware* de autenticação verifique se o usuário tem permissão dentro do *tenant* (por exemplo, checando as *claims* do JWT) sem que cada *controller* precise repetir essa lógica (02:42:55).

- **Recomendação de Ferramenta para BFF/Proxy Reverso**: Foi recomendada a biblioteca YARP (Yet Another Reverse Proxy) para construir um Backend-for-Frontend (BFF) utilizando ASPNET Core. O BFF é um padrão que reduz o acoplamento do *front-end* com múltiplos serviços e pode ser usado para adicionar um *proxy* reverso com capacidade de transformar e rotear as requisições (02:48:01).
- **Discussão sobre Ferramentas e Frontend**: Thiago Borba notou que o frontend é totalmente "vai coach". Gabriel kohlrausch expressou esperança de que as interfaces do sistema se tornem agênicas, eliminando telas ricas e complexas, de modo que só seria necessário um chat para voz e áudio, sem botões, CSS ou HTML. Farley Souza concordou que CSS é "terrível", pois "quando ele não mata, ele aleija" (02:56:00).
- **Comentários sobre CSS e Design**: Gabriel kohlrausch mencionou que tinha um trauma com CSS, e tentou se lembrar do nome de uma figura de autoridade brasileira em CSS. Thiago Borba argumentou que o mundo era mais simples, mesmo que não fosse tão bonito, o que levou a uma discussão sobre a subjetividade da beleza e da arte (02:57:12). Farley Souza observou que a arte é subjetiva (02:58:02).
- **Recomendação de Leitura e Autoridade Técnica**: Gabriel kohlrausch relembrou o nome do livro e o autor, o "Maor," recomendando o livro "Writing Soft" como uma leitura que pode fazer as pessoas questionarem tudo o que fazem. Ele descreveu o autor como alguém com grande autoridade e background que lhes permite ter um posicionamento firme sobre "o jeito certo" de fazer as coisas. Gabriel kohlrausch incentivou as pessoas a assistirem palestras e procurarem material do autor no YouTube para refletir sobre os temas (02:58:02).
- **Encerramento da Reunião e Agradecimentos**: Gabriel kohlrausch concluiu que nada havia sido resolvido, mas tudo estava certo, e agradeceu a todos que participaram, expressando satisfação pela troca de ideias. Ele informou que o próximo encontro seria com Francisco ou Bruno na próxima semana, e se colocou à disposição para contato via grupo ou WhatsApp, embora não garantisse resposta imediata, mas assegurou que responderia, mesmo que demorasse um pouco (02:59:17). Ele reforçou que as portas estão abertas para sugestões, conversas e dúvidas, incentivando a proximidade com o grupo (03:00:47).

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com

- **Confirmação de Atividades no Feriado**: Anderson Perim perguntou a Gabriel kohlrausch se haveria reunião na segunda-feira do feriado, já que era um feriado na terça-feira. Gabriel kohlrausch expressou que estava tão ocupado que nem se lembrava do feriado, mas disse que confirmaria e faria uma enquete para verificar se haveria quórum para realizar o encontro (03:01:37). Por fim, Gabriel kohlrausch desejou a todos um bom descanso e bons deploys até o final da semana (03:00:47) (03:02:13).

Revise as anotações do Gemini para checar se estão corretas. Confira dicas e saiba como o Gemini faz anotações

*Como está a qualidade de **destas observações?** Responda a uma breve pesquisa para nos dar seu feedback, incluindo o quanto as observações foram úteis para o que você precisa.*

Conteúdo licenciado para Farley de Souza Rufino - farley.t.i@hotmail.com